qPCRtools: a cross-platform desktop application for reproducible qPCR analysis without programming

Xiang Li[1,2,*] Huichuan Huang[1,2,*]

[1] State Key Laboratory for Conservation and Utilization of Bio-Resources in Yunnan, Yunnan Agricultural University, Kunming, China

[2] Key Laboratory for Agro-Biodiversity and Pest Control of Ministry of Education, Yunnan Agricultural University, Kunming, China

[*] Corresponding authors: lixiang117423@gmail.com (X.L.), absklhhc@gmail.com (H.H.)

Present address (X.L.): Yazhouwan National Laboratory, Sanya, Hainan, China

Abstract

Summary: Quantitative real-time PCR (qPCR) is a standard technique for measuring gene expression, yet turning instrument-exported C_q values into publication-ready results requires several non-trivial steps: estimating amplification efficiency, choosing a quantification method, testing groups statistically, and preparing figures. Established tools for this chain are predominantly programming-based—including our own R package *qPCRtools*—and remain out of reach for students and wet-lab researchers without programming experience, whereas instrument software, free stand-alone programs and web applications each cover only part of the workflow or tie the analysis to one platform. We present *qPCRtools*, a cross-platform desktop application written in C++17 with Qt6 that exposes the analytical core of the R package through a point-and-click interface and ships as a single executable requiring no R, Python or other runtime. It implements standard-curve construction and efficiency estimation, the ΔC_t, $\Delta\Delta C_t$ and efficiency-based (RqPCR) quantification methods, group comparison by Welch t-test, Wilcoxon rank-sum test or one-way ANOVA with Tukey HSD letters, quality-control warnings, in-interface figures, and a plain-text record of the inputs and parameters of each analysis. The implementation was validated against an independent base-R reference for every quantification method. Preparing the port also revealed and corrected several defects in result aggregation, each covered by a regression test.

Availability and Implementation: *qPCRtools* is written in C++17 with Qt6 and built with CMake; the core algorithm library requires only Qt Core and optionally links Eigen3, GSL and OpenXLSX. The source code is available under the MIT license at `https://github.com/lixiang117423/qPCRtools_cpp`, with pre-built binaries for version 1.3.0 in the repository releases; no registration is required. Linux build instructions are provided, but no Linux binary is released. The software and its source will remain available for at least two years.

Contact: lixiang117423@gmail.com; absklhhc@gmail.com

Supplementary Information: Supplementary Figures S1–S2 give the full analysis output and the validation comparison; Supplementary Tables S1–S3 summarise numerical agreement, the sensitivity of the group comparison to the analysis scale and a reproducible re-analysis of published qPCR data.

1 Introduction

Quantitative real-time PCR (qPCR) is among the most widely used techniques in molecular biology for quantifying nucleic acids and measuring gene expression [5]. Current reporting guidance is provided by MIQE 2.0 [6]. A sound analysis involves several interdependent steps:

estimating primer amplification efficiency, choosing a quantification strategy, testing groups statistically, and preparing figures [4]. When target and reference efficiencies are comparable, the $2^{-\Delta\Delta C_T}$ method [10] is convenient; when they differ, an efficiency-corrected or standard-curve-based approach is required [14]. These choices, combined with the need for rigorous statistics, make qPCR analysis demanding for many laboratory researchers.

A range of tools cover different parts of this workflow (Table 1). Scripting-based implementations—our own *qPCRtools* [9] and *pcr* [1]—implement the calculation chain transparently but require programming skill and a configured environment. Software distributed with real-time instruments (for example QuantStudio Design & Analysis and CFX Maestro) offers a graphical workflow but is tied to that instrument. Free stand-alone programs cover single steps: REST [15] computes efficiency-based relative expression for two-group comparisons with a randomisation test, LinRegPCR [16, 19] estimates baseline and efficiency from raw fluorescence, and qBase [7] implements relative quantification with the geNorm stability measure [21]. Free web applications—SATQPCR [17], DAG Expression [3] and Auto-qPCR [11]—integrate several steps but require uploading the data to a remote server or deploying the web stack locally. The gap is therefore not a missing algorithm but a missing delivery mode: *auditable numerical implementation* combined with a *native, cross-platform, zero-dependency* graphical workflow.

Table 1: Delivery mode and functionality of representative qPCR analysis tools. "Free for non-commercial use" means no purchase is required for academic users; software distributed with an instrument is marked as not free. "Reference-gene stability" denotes computational selection or ranking of reference genes, not merely normalisation to a chosen reference gene. • provided, ○ not provided, ○/• partial or version-dependent (statistics were added in the later qBase+ release; vendor software differs between instruments and versions). Compiled from the cited papers, the vendors' documentation and the survey of Pabinger *et al.* [12]. [a] *qPCRtools* [9], *pcr* [1], qpcR [18], NormqPCR [13], and the archived Bioconductor package EasyqpcR [8]. [b] QuantStudio Design & Analysis, CFX Maestro. [c] SATQPCR [17], DAG Expression [3], Auto-qPCR [11].

Tool	Free	No coding	Offline	Efficiency	Statistics	Ref.-gene stability
R packages[a]	•	○	•	•	○/•	•
This application	•	•	•	•	•	•
REST	•	•	•	•	•	○
LinRegPCR	•	•	•	•	○	○
qBase	•	•	•	•	○	•
qBase+	○	•	•	•	•	•
Instrument software[b]	○	•	•	•	○/•	○
Web apps[c]	•	•	○	•	•	•

Statement of need

The programming requirement is not a marginal obstacle. In our own teaching and collaborative work, the users who most need a defensible qPCR analysis are frequently those least able to install an R environment and write the few lines of code the analysis requires: newly enrolled students, visiting researchers, and wet-lab staff whose projects run for a single season. They fall back either on manual spreadsheet calculations—difficult to audit and easy to get wrong—or on whichever instrument software happens to be installed. Our R package [9] provides the algorithms, but its use is bounded by the R environment itself.

As a pragmatic test of the workflow, we re-analysed the two real example plates distributed with the 2022 *qPCRtools* paper: one four-fold dilution series and one separate treatment plate with four biological replicates. Both analyses were completed through the graphical interface and exported with a parameter record, without a spreadsheet or custom script (Supplementary Table S3). This is a reproducible published-data re-analysis, not a controlled human-factors trial.

We therefore reimplemented the analytical core as a native desktop application that implements the same three quantification methods while removing every prerequisite except a double-click. *qPCRtools* makes three contributions: (i) a C++/Qt architecture in which a tested core algorithm library is decoupled from a web-based user interface; (ii) a reproducible validation against an independent base-R reference across all quantification methods; and (iii) statistical and functional extensions beyond the R version (non-parametric tests, effect sizes, outlier removal, quality-control warnings, automatic reference-gene selection). Preparing the port also revealed defects in result aggregation shared with the R package—the input of the reference-gene stability measure, replicate handling in the efficiency method, and the $\Delta\Delta C_t$ calibrator—as well as one specific to the C++ code (multiple-comparison P-values); all were corrected and are covered by regression tests.

2 Implementation

2.1 Architecture

qPCRtools is written in C++17 and built on Qt6 [20]. The user interface is rendered with web technologies—Bootstrap 5.3 for layout and Apache ECharts [2] for figures—inside a `QWebEngineView`, and exchanges data with the native backend through Qt WebChannel. All numerical computation therefore stays in compiled code while the presentation layer remains plain HTML, CSS and JavaScript. The analytical core is a standalone static library, `qpcr_core`, that depends only on Qt Core (plus optional Eigen and GSL) and is independent of the GUI, so it is exercised directly by the unit tests and by the command-line validation driver. Figure 1 shows the interface at the end of a complete analysis.

2.2 Quantification algorithms

The core reimplements the methods of the R package. Amplification efficiency is obtained by ordinary least squares of C_q on $\log_d$(concentration), i.e. the number of d-fold dilution steps (d = dilution factor, 4 in the bundled example), yielding slope s, intercept, R^2 and a regression P-value; efficiency is

$$E = d^{-1/s} - 1, \tag{1}$$

with s expressed per $\log_d$ unit (equivalently, per dilution step). Relative expression is computed by the ΔC_t and $\Delta\Delta C_t$ methods [10],

$$\mathrm{RE} = 2^{-\left[(C_{q,\mathrm{target}} - C_{q,\mathrm{ref}})_{\mathrm{treated}} - \overline{(C_{q,\mathrm{target}} - C_{q,\mathrm{ref}})}_{\mathrm{control}}\right]}, \tag{2}$$

where the first term is sample-level and the overbar is the mean ΔC_t across control biological replicates. Relative expression is also computed by an efficiency-based (RqPCR) method [17] in which per-sample quantities are $E^{C_{q,\min} - C_q}$, normalized by the geometric mean of one or more reference genes [14]. In this method $C_{q,\min}$ is obtained separately within each biological replicate across the treatments, so the calculation is a within-block comparison rather than a general unpaired design. Efficiencies are supplied per gene in the design table (or estimated by the built-in standard-curve module).

2.3 Statistical analysis

Groups are compared on the linear relative-expression scale, which is the scale on which values are reported and plotted. Because the linear and ΔC_t scales can give different P values for the same data, the ΔC_t-scale comparison is reported as a sensitivity analysis in Supplementary Table S2. The core library implements Welch and Student two-sample t-tests, the paired t-test, Wilcoxon rank-sum and signed-rank tests, one-way ANOVA with Tukey HSD letters, Cohen's d,

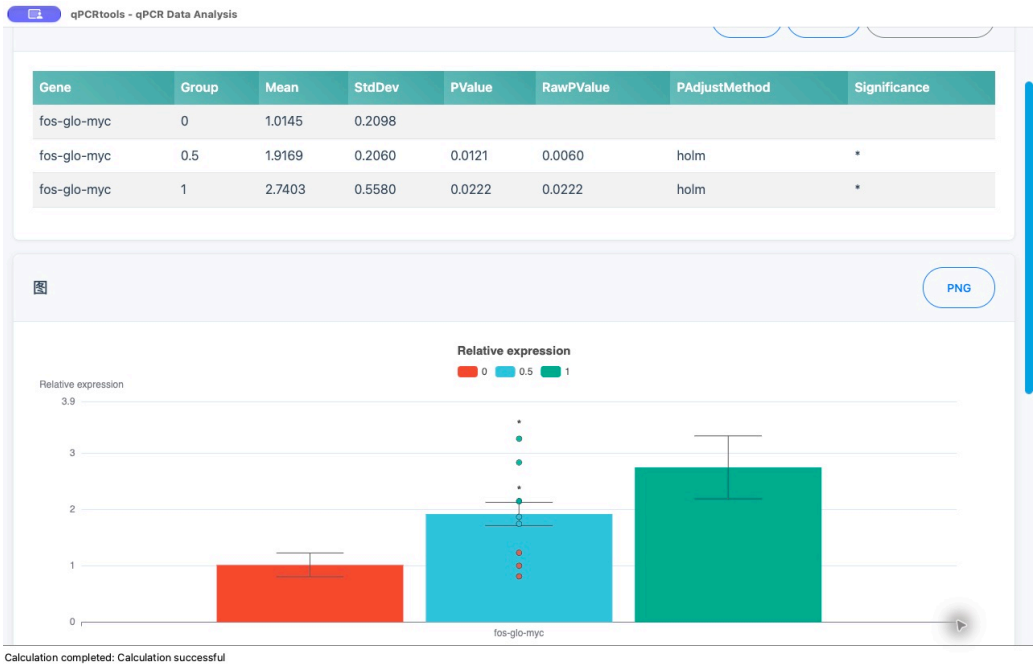


Figure 1: The *qPCRtools* desktop interface (screen capture of version 1.3.0). The upper panel is the summary table from the bundled example—the target gene *fos-glo-myc* normalised to *Beta Actin*, three dose groups with three biological replicates each—analysed by the $\Delta\Delta C_t$ method with Welch tests against the control. It reports the mean, standard deviation, Holm-adjusted and raw P values, adjustment method and significance symbol. The lower panel redraws the group means with standard-deviation error bars, individual biological replicates and Holm-adjusted symbols; the error bars are not repeated as a legend entry. The interface is bilingual (Chinese/English); the English language is shown here.

IQR-based outlier removal and automatic reference-gene selection by geNorm [21]; the interface exposes the Welch t-test, the rank-sum test and ANOVA with Tukey HSD, while the paired tests are covered by the test suite but not yet offered in the interface. When several groups are compared with the same control, Welch and rank-sum P values are adjusted by Holm's method within each gene, and the unadjusted value is retained in the exported table. ANOVA uses Tukey HSD. Correction is not applied across genes, which we state as a limitation.

2.4 Software engineering and scope

Results are shown as tables that can be exported as CSV or Excel, with Tukey letters in the table. The application renders the figures in the interface: group means with standard-deviation error bars and significance labels, and the fitted standard curves over the measured dilution series, both exportable as PNG images. Each analysis can also be exported as a plain-text *analysis record* listing the input files and their row counts, the reference gene(s), the control group, the statistical test, the outlier-removal setting, the software version and a timestamp, so that a reader can re-run the analysis without inspecting the session. Quality control is reported as warnings rather than decisions: wells with C_q above a configurable limit (default 35), samples whose technical replicates differ by more than a configurable standard deviation (default 0.5 C_q), and primers with efficiency outside 90–110% are listed on the results page and repeated in the analysis record, while no data are removed automatically. MIQE 2.0 requires these quantities to be reported rather than to satisfy fixed cut-offs [6], so the defaults are meant to be adapted to the instrument and assay. *qPCRtools* is managed with CMake and ships with unit tests; the interface is bilingual and reads CSV/TXT input, and Excel (`.xlsx`) workbooks when built with the optional OpenXLSX component (the released binaries read CSV/TXT). It is distributed as a single self-contained executable for Windows and macOS, requiring no R, Python or runtime installation. The macOS binary is built for Apple Silicon, requires macOS 12 or newer, and is ad-hoc signed but not notarized. Linux builds are supported by the source and build scripts but are not part of the released binary set. Two R functions are not yet ported—the standard-curve slope–intercept quantification (`CalExpCurve`) and the reverse-transcription reagent volume calculation (`CalRTable`)—and are planned for a future release.

3 Results

3.1 Typical analysis workflow

We applied the application to the bundled example dataset (`examples/cq.csv`, `examples/design.csv`): the target gene *fos-glo-myc* against the reference gene *Beta Actin* across three treatment groups (0, 0.5, 1) with three biological replicates each. With the $\Delta\Delta C_t$ method and Welch tests against the control, the mean relative expression was 1.01, 1.92 and 2.74 in the three groups ($n = 3$ biological replicates). The comparisons against the control had unadjusted $P = 0.0060$ and $P = 0.0222$; after Holm adjustment they were $P = 0.0121$ and $P = 0.0222$. As defined by Livak and Schmittgen, the calibrator is the control group. These values are reproduced without the application by `validation/check_example_stats.R`, and the complete output is shown in Supplementary Figure S1B. On a four-fold dilution series of four genes, the standard-curve module recovered slopes of -2.07 to -2.24 C_q per dilution step. Three curves had $R^2 > 0.99$ and efficiencies of 0.85–0.94; Gene1 had $R^2 = 0.75$ and was correctly flagged as unsuitable. The default 90–110% rule additionally flagged Gene3 (85.5%) and Gene4 (87.3%), and the quality-control panel flagged the single $C_q = 40$ well of Gene1.

In a second analysis of the two published example plates distributed with the 2022 *qPCRtools* paper, we used the dilution series to fit four curves and the separate treatment plate to compare five target genes with *OsUBQ* in two groups with four biological replicates. Gene1 failed quality control ($R^2 = 0.7509$), the other curves had $R^2 = 0.9983$–0.9993, and the supplied gene-specific

efficiencies selected standard-curve expression rather than $2^{-\Delta\Delta C_t}$. Welch tests with Holm adjustment gave $P = 0.0368$ for *OsPR10* and $P = 0.00230$ for *OsWAK91*; the other three genes were not significant. The workflow exported summary and replicate-level tables, both figures and a parameter record, and is reproduced by a deterministic checker (Supplementary Table S3). The two plates are separate published examples and are not presented as one biological experiment; the complete workflow—import, method selection, analysis and export—was performed in the graphical interface without writing code.

3.2 Validation against an independent reference

We compared the application with an independent base-R implementation of the three quantification methods—$\Delta\Delta C_t$, the standard curve and the efficiency-based RqPCR method—using the same on-disk inputs (Supplementary Table S1 and Figure S2). The reference was computed directly from the raw C_q tables and did not call the R package. Across 58 matched quantities, expression values, standard deviations, standard errors, raw and Holm-adjusted P values, slopes, intercepts, R^2 values and efficiencies agreed to floating-point precision (maximum relative difference 1.9×10^{-14}). Every point falls on the line $y = x$.

The validation is reproducible from the open scripts in `validation/` (see Availability). Numerical agreement establishes consistency with the specified algorithms, not biological correctness, and the choice of scale remains a design decision. Preparing the port also revealed defects in result aggregation, including the input of the reference-gene stability measure, replicate handling in the efficiency method and the $\Delta\Delta C_t$ calibrator, as well as multiple-comparison P values in the C++ code. Each was corrected and is covered by a regression test. Earlier GUI releases contained these defects and should be re-run with version 1.3.0. The corresponding R package is available as version 2.0.1, and the CRAN-compatible 1.0.2 source has been submitted to CRAN. A retrospective assessment of the 2022 paper's example analyses is provided with the R-package release.

4 Availability and implementation

Implementation: *qPCRtools* is a C++17/Qt6 desktop application built with CMake (≥ 3.20); the core algorithm library requires only Qt Core and optionally links Eigen3, GSL and OpenXLSX, while the interface requires Qt WebEngine and WebChannel. It is distributed as a single self-contained executable for Windows and macOS—no R, Python or other runtime, and no registration, are needed to download or run it. The macOS binary is built for Apple Silicon, requires macOS 12 or newer, and is ad-hoc signed but not notarized. Linux builds are supported by the source and build scripts but are not part of the released binary set.

Availability: the source code is freely available under the MIT license at `https://github.com/lixiang117423/qPCRtools_cpp`. Pre-built binaries for version 1.3.0 and the matching source tag are in the repository releases. The repository includes documentation, a unit-test suite, example datasets, the published-data re-analysis used in this paper and a fully reproducible validation package (`validation/`) comprising a base-R reference, a C++ command-line driver linked against `qpcr_core`, and a comparison script that regenerates all validation outputs and figures. We will keep the software and its source code available for at least two years after publication. Supplementary Figures S1–S2 and Supplementary Tables S1–S2 are available online only.

Authors' contributions

XL and HH designed this study and interpreted the results. XL developed the software, performed the validation and wrote the manuscript. Both authors take responsibility for the con-

tent.

Acknowledgements

The authors thank the developers of the R packages and libraries on which this work builds, and the users who provided feedback during development.

Funding

This work was supported by the National Key R&D Program of China (2021YFD1000200), the National Natural Science Foundation of China (31801792, 31960554), and the Fok Ying Tung Education Foundation.

Conflict of interest

The authors declare no competing interests.

Use of AI-assisted tools

Generative AI tools were used during different stages of the work. Claude Code (Anthropic) assisted with code development, testing, validation scripts and early manuscript drafting between March and June 2026. OpenAI Codex assisted with code refactoring, additional tests, documentation and manuscript revision between August and October 2026. All AI-assisted changes were reviewed by the authors, who ran the tests and analyses, checked the numerical results and take responsibility for the content. The tools are not authors.

References

[1] Mohaned Ahmed and Dae-Ryong Kim. pcr: an R package for quality assessment, analysis and testing of qPCR data. *PeerJ*, 6:e4473, 2018.

[2] Apache Software Foundation. Apache ECharts: A declarative framework for rapid construction of web-based visualization. `https://echarts.apache.org/`, 2024.

[3] Maria Ballester, Raquel Cordón, and Josep M. Folch. DAG Expression: high-throughput gene expression analysis of real-time PCR data using standard curves for relative quantification. *PLoS ONE*, 8(11):e80385, 2013.

[4] Stephen A. Bustin, Jean-François Beaulieu, Jim Huggett, Ramnath Jaggi, Frederick S. B. Kibenge, Pal A. Olsvik, Lois C. Pennington, and Tobias Toegl. MIQE précis: practical implementation of minimum standard guidelines for fluorescence-based quantitative real-time PCR experiments. *BMC Molecular Biology*, 11:74, 2010.

[5] Stephen A. Bustin, Vladimir Benes, Jeremy A. Garson, Jan Hellemans, Jim Huggett, Mikael Kubista, Reinhold Mueller, Tania Nolan, Michael W. Pfaffl, Gregory L. Shipley, et al. The MIQE guidelines: minimum information for publication of quantitative real-time PCR experiments. *Clinical Chemistry*, 55(4):611–622, 2009.

[6] Stephen A. Bustin, Jan M. Ruijter, Maurice J. B. van den Hoff, Mikael Kubista, Michael W. Pfaffl, Gregory L. Shipley, Nham Tran, Stefan Rödiger, Andreas Untergasser, Reinhold Mueller, Tania Nolan, Mojca Milavec, Malcolm J. Burns, Jim F. Huggett, Jo Vandesompele, and Carl T. Wittwer. MIQE 2.0: revision of the minimum information for publication of

quantitative real-time PCR experiments guidelines. *Clinical Chemistry*, 71(6):634–651, 2025.

[7] Jan Hellemans, Geert Mortier, Anne De Paepe, Frank Speleman, and Jo Vandesompele. qBase relative quantification framework and software for management and automated analysis of real-time quantitative PCR data. *Genome Biology*, 8(2):R19, 2007.

[8] Sylvain Le Pape. EasyqpcR: for low-throughput real-time quantitative PCR data analysis. Bioconductor package, 2017. Archived Bioconductor package.

[9] Xiang Li, Yingmin Wang, Jingyu Li, Xinyue Mei, Yixiang Liu, and Huichuan Huang. qPCRtools: an R package for qPCR data processing and visualization. *Frontiers in Genetics*, 13:1002704, 2022.

[10] Kenneth J. Livak and Thomas D. Schmittgen. Analysis of relative gene expression data using real-time quantitative PCR and the $2^{-\Delta\Delta C_T}$ method. *Methods*, 25(4):402–408, 2001.

[11] Gilles Maussion, Rhalena A. Thomas, Iveta Demirova, Gracia Gu, Eddie Cai, Carol X.-Q. Chen, Narges Abdian, Theodore J. P. Strauss, Sabah Kelaï, Angela Nauleau-Javaudin, Lenore K. Beitel, Nicolas Ramoz, Philip Gorwood, and Thomas M. Durcan. Auto-qPCR; a Python-based web app for automated and reproducible analysis of qPCR data. *Scientific Reports*, 11:21293, 2021.

[12] Stephan Pabinger, Stefan Rödiger, Anja Kriegner, Klemens Vierlinger, and Andreas Weinhäusel. A survey of tools for the analysis of quantitative PCR (qPCR) data. *Biomolecular Detection and Quantification*, 1:23–33, 2014.

[13] James R. Perkins, John M. Dawes, Steve B. McMahon, David L. H. Bennett, Christine Orengo, and Matthias Kohl. ReadqPCR and NormqPCR: R packages for the reading, quality checking and normalisation of RT-qPCR quantification cycle (Cq) data. *BMC Genomics*, 13:296, 2012.

[14] Michael W. Pfaffl. A new mathematical model for relative quantification in real-time RT-PCR. *Nucleic Acids Research*, 29(9):e45, 2001.

[15] Michael W. Pfaffl, Graham W. Horgan, and Leo Dempfle. Relative expression software tool (REST) for group-wise comparison and statistical analysis of relative expression results in real-time PCR. *Nucleic Acids Research*, 30(9):e36, 2002.

[16] Christian Ramakers, Jan M. Ruijter, Ronald H. Lekanne Deprez, and Antoon F. M. Moorman. Assumption-free analysis of quantitative real-time polymerase chain reaction (PCR) data. *Neuroscience Letters*, 339(1):62–66, 2003.

[17] Corinne Rancurel, Thuy Van Tran, Corinne Elie, and Frédérique Hilliou. SATQPCR: website for statistical analysis of real-time quantitative PCR data. *Molecular and Cellular Probes*, 46:101418, 2019.

[18] Christian Ritz and Andrej-Nikolai Spiess. qpcR: an R package for sigmoidal model selection in quantitative real-time polymerase chain reaction analysis. *Bioinformatics*, 24(13):1549–1551, 2008.

[19] Jan M. Ruijter, Christian Ramakers, Willem M. H. Hoogaars, Yann Karlen, Olaf Bakker, Maurice J. B. van den Hoff, and Antoon F. M. Moorman. Amplification efficiency: linking baseline and bias in the analysis of quantitative PCR data. *Nucleic Acids Research*, 37(6):e45, 2009.

[20] The Qt Company. Qt Cross-Platform Application Development Framework. `https://www.qt.io/`, 2024. Version 6.x.

[21] Jo Vandesompele, Katleen De Preter, Filip Pattyn, Bruce Poppe, Nicole Van Roy, Anne De Paepe, and Frank Speleman. Accurate normalization of real-time quantitative RT-PCR data by geometric averaging of multiple internal control genes. *Genome Biology*, 3(7):research0034, 2002.